# Progress Report 3: PM2.5 Prediction ML System

**วิชา:** ML Systems
**หัวข้อโปรเจกต์:** FoonAlert – Bangkok PM2.5 Prediction & Alert System
**วันที่:** เมษายน 2026

---

## 1. Project Overview (สรุปย่อ)

**FoonAlert** คือระบบ Machine Learning สำหรับพยากรณ์ค่า PM2.5 (μg/m³) รายวันล่วงหน้า โดยใช้ข้อมูลรายชั่วโมงจากสถานีตรวจวัดคุณภาพอากาศในกรุงเทพมหานคร เพื่อช่วยให้ประชาชนวางแผนกิจกรรมกลางแจ้งและหน่วยงานสาธารณสุขออกคำเตือนล่วงหน้าได้อย่างมีประสิทธิภาพ

**Use Case & ปัญหา:**

- ปัญหามลพิษทางอากาศ PM2.5 ในกรุงเทพฯ ส่งผลกระทบต่อสุขภาพ โดยเฉพาะกลุ่มเปราะบาง (ผู้สูงอายุ เด็กเล็ก ผู้ป่วยทางเดินหายใจ)
- ต้องการระบบเตือนล่วงหน้าที่แม่นยำ เพื่อลดความเสี่ยงด้านสุขภาพ

**ML Task:** Time-Series Regression — ทำนายค่า PM2.5 ต่อเนื่องจากข้อมูลในอดีต

**Dataset:**

- **ข้อมูลหลัก (ใหม่ใน Progress 3):** `station_145_long.csv` — สถานี 145 เขตบางขุนเทียน ข้อมูลรายชั่วโมง 25,714 rows (มี.ค. 2023 – มี.ค. 2026) ถูกแปลงเป็นค่าเฉลี่ยรายวัน 1,076 วัน
- **ข้อมูลเสริม:** `PM2.5(2024).xlsx` และ `PM2.5(2025).xlsx` — ข้อมูลสถานี 10T (เขตบางกะปิ) ที่เคยใช้ใน Progress 1–2
- **ข้อมูลขนาดใหญ่:** AirBKK scraped dataset รวม ~21.9 ล้าน rows (86 สถานี × 13 parameters)

**โมเดลที่เลือกใช้:** Random Forest — เป็น Champion model ทั้งบนข้อมูลสถานี 10T (MAE 4.55) และสถานี 145 (MAE 4.57) ให้ผลดีกว่า LSTM, XGBoost, Ridge, และ Linear Regression

**สถานะปัจจุบัน:**

- Pipeline ครบ: data → preprocessing → feature engineering → training → evaluation → deployment
- อัพเดทผลลัพธ์ด้วย input ใหม่ Station 145 (เขตบางขุนเทียน)
- FastAPI service ให้บริการผ่าน REST API ( `/predict` , `/actual` , `/retrain` )
- Docker Compose stack (PostgreSQL + MLflow + Airflow + API)
- Automated monitoring + retraining (MAE/PSI-based drift detection)
- ONNX model export สำหรับ fast inference

**ผลการทดสอบล่าสุด — Station 145 เขตบางขุนเทียน (Test Set: 2025–2026):**

| Model | MAE ↓ | RMSE ↓ | $R^2$ ↑ |
|---|---|---|---|

| Model | MAE ↓ | RMSE ↓ | $R^2$ ↑ |
|---|---|---|---|
| Linear Regression (Baseline) | 4.5986 | 6.6231 | 0.7512 |
| Ridge Regression | 4.5984 | 6.6229 | 0.7512 |
| **Random Forest** | **4.5713** | **6.5123** | **0.7594** |
| XGBoost | 4.5776 | 6.6307 | 0.7506 |

> **หมายเหตุ:** LSTM ผ่านการทดสอบบนสถานี 10T ใน Progress 2 แล้ว (MAE 6.22) ให้ผลแย่กว่า Random Forest จึงเลือก Random Forest เป็น Champion model

**เปรียบเทียบผลลัพธ์ระหว่างสถานี:**

| Dataset | Station | Train Period | Test Period | Best Model | MAE |
|---|---|---|---|---|---|
| PM2.5(2024/2025).xlsx | 10T (บางกะปิ) | 2024 | 2025 | Random Forest | 4.55 |
| station_145_long.csv | 145 (บางขุนเทียน) | 2023–2024 | 2025–2026 | Random Forest | **4.57** |

→ ผลลัพธ์สอดคล้องกัน: Random Forest ให้ MAE ~4.5–4.6 μg/m³ ทั้งสองสถานี แสดงให้เห็นว่า pipeline มีความ robust และ generalize ได้ดีข้ามสถานี

---

# 2. MLOps & Deployment / CI-CD Orchestration

## 2.1 Workflow Overview

```
┌──────────────────────────────────────────────────────────────────────┐
│                      FoonAlert ML Workflow                           │
│                                                                      │
│  ┌──────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────┐ │
│  │  Data    │──►│ Preprocessing│──►│ Feature      │──►│ Training │ │
│  │ Ingestion │    │ (ffill,      │    │ Engineering  │    │ (5 models│ │
│  │ (CSV/API) │    │  outliers)   │    │ (17 features│    │  GridCV) │ │
│  └──────────┘    └──────────────┘    └──────────────┘    └────┬─────┘ │
│                                                               │       │
│  ┌──────────┐    ┌──────────────┐    ┌──────────────┐    ┌────▼─────┐ │
│  │ Monitor  │◄──│  Deployment  │◄──│ ONNX Export  │◄──│Evaluation│ │
│  │ (MAE/PSI)│    │  (FastAPI)   │    │              │    │(MAE/RMSE)│ │
│  └────┬─────┘    └──────────────┘    └──────────────┘    └──────────┘ │
│       │                                                              │
│       └── Threshold exceeded? ──YES──► Trigger Retrain DAG           │
└──────────────────────────────────────────────────────────────────────┘
```

## 2.2 ขั้นตอนของระบบ

| ขั้นตอน | รายละเอียด | สถานะ |
|---|---|---|
| **Data Ingestion** | โหลดข้อมูลจาก CSV (station_145_long.csv) หรือ Excel; แปลงรายชั่วโมงเป็นรายวัน | Automated |
| **Preprocessing** | Forward-fill missing values, ลบ outlier (0–500 μg/m³ range) | Automated |

| ขั้นตอน | รายละเอียด | สถานะ |
|---|---|---|
| **Feature Engineering** | สร้าง 17 features: lag (1,2,3,5,7), rolling mean/std (3,7,14 วัน), time features, change features | Automated |
| **Training** | GridSearchCV + TimeSeriesSplit สำหรับ 5 โมเดล (LR, Ridge, RF, XGBoost, LSTM) | Automated via Airflow DAG |
| **Evaluation** | MAE, RMSE, $R^2$ บน test set (temporal split) | Automated |
| **ONNX Export** | แปลงทุกโมเดลเป็น ONNX สำหรับ fast inference | Automated |
| **Deployment** | FastAPI service — `/predict`, `/actual`, `/retrain` | Running in Docker |
| **Monitoring** | Rolling MAE (30 วัน) + PSI drift detection | Automated (daily Airflow DAG) |
| **Retrain Trigger** | MAE > 6.0 μg/m³ หรือ PSI > 0.2 → trigger training DAG | Automated |
| **Model Promotion** | เปรียบเทียบ challenger vs champion (MAE improvement ≥ 5%) | Manual (planned for automation) |
| **Feature Store Update** | เพิ่ม features ใหม่ (meteorological variables) | Manual |

## 2.3 Data Drift Detection

ระบบใช้ 2 metrics หลักในการตรวจจับ drift (implement ใน `src/monitor.py`):

1. **Rolling MAE (30-day window):** ถ้า MAE > 6.0 μg/m³ → trigger retrain
2. **PSI (Population Stability Index):** เปรียบเทียบ distribution ของ predicted vs actual
   - PSI < 0.1 → **Stable** (ปกติ)
   - PSI 0.1–0.2 → **Moderate shift** (เฝ้าระวัง)
   - PSI > 0.2 → **Significant shift** → trigger retrain

Monitoring DAG ( `pm25_pipeline_dag.py` ) รันอัตโนมัติทุกวัน 01:00 UTC ตรวจทั้ง MAE และ PSI แล้ว branch ตามเงื่อนไข

## 2.4 Model Deployment & Optimization

**รูปแบบ Deployment:** Batch Prediction (Periodic) ผ่าน FastAPI + Docker

**เหตุผลที่เลือก Batch:**

- PM2.5 เปลี่ยนแปลงในระดับชั่วโมง ไม่ต้องการ sub-second latency
- การทำนาย 24 ชั่วโมงล่วงหน้าเหมาะกับ pre-compute + cache
- ประหยัด compute cost เมื่อเทียบกับ real-time streaming

**Model Optimization:**

- **ONNX Export:** ทุกโมเดลถูก convert เป็น `.onnx` สำหรับ inference ที่เร็วขึ้น (ไม่ต้องโหลด scikit-learn ทั้งหมด)
- **ONNX Runtime:** `src/predict_onnx.py` ใช้ `onnxruntime.InferenceSession` สำหรับ production inference
- **Model Size Optimization:** Random Forest (champion) export เป็น ONNX ลดขนาดไฟล์และ load time

### 2.5 ส่วนที่ยัง Manual และแนวทางพัฒนาในอนาคต

| ส่วนที่ยัง Manual | แนวทางพัฒนา |
|---|---|
| Feature engineering ใหม่ | AutoML / feature store |
| Data quality anomaly detection | Sensor health monitoring DAG |
| Multi-station expansion (5-10 stations) | Parameterized pipeline per station_id |

---

# 3. Model Serving

## 3.1 รูปแบบการใช้งาน

ระบบให้บริการผ่าน **REST API (FastAPI)** โดยมี 4 endpoints หลัก:

| Endpoint | Method | Input | Output | ใช้งาน |
|---|---|---|---|---|
| `/health` | GET | — | `{"status": "ok"}` | Liveness check |
| `/model/info` | GET | — | ชื่อโมเดล + feature list | Metadata |
| `/predict` | POST | ≥15 วัน PM2.5 history | predicted PM2.5 วันถัดไป | **Core prediction** |
| `/actual` | POST | date + pm25_actual | matched prediction + error | บันทึกค่าจริง |
| `/retrain` | POST | threshold (optional) | MAE + retrain status | Trigger retraining |

## 3.2 Input / Output

**Input (POST /predict):**

```
{
  "history": [
    {"date": "2025-03-01", "pm25": 28.5},
    {"date": "2025-03-02", "pm25": 32.1},
    {"date": "2025-03-03", "pm25": 45.0},
    ...
    {"date": "2025-03-15", "pm25": 38.7}
  ]
}
```

- ต้องส่งข้อมูลอย่างน้อย **15 วันย้อนหลัง** (เนื่องจากใช้ lag 7 + rolling 14)
- แต่ละรายการมี `date` (YYYY-MM-DD) และ `pm25` (0–500 μg/m³)

**Output:**

```
{
  "prediction_date": "2025-03-16",
  "predicted_pm25": 36.42,
  "unit": "μg/m³",
  "model": "random_forest"
}
```

### 3.3 การทดลองใช้งานจริง

**Pipeline Run — Station 145 (เขตบางขุนเทียน):**

ทดลองรัน pipeline กับข้อมูลจริงจากสถานี 145 (station_145_long.csv):

```
============================================================
STATION 145 — FULL PIPELINE
============================================================
Station 145 daily data: 1,076 days
Date range: 2023-03-29 to 2026-03-19
Missing values: 2

Train: 640 days (2023-03-29 to 2024-12-31)
Test:  436 days (2025-01-01 to 2026-03-19)

=== Train Data Preprocessing ===
  Missing values: 2 → 0 (method=ffill)
  Final shape: (640, 2)

=== Test Data Preprocessing ===
  Missing values: 0 → 0
  Final shape: (436, 2)

Features (17): pm25_lag_1, pm25_lag_2, pm25_lag_3, pm25_lag_5,
  pm25_lag_7, pm25_rolling_mean_3/7/14, pm25_rolling_std_3/7/14,
  day_of_week, month, day_of_year, is_weekend,
  pm25_diff_1, pm25_pct_change_1

X_train: (633, 17), X_test: (429, 17)
```

**ผลลัพธ์:**

| Model | MAE (μg/m³) | RMSE | $R^2$ | Best Hyperparams |
|---|---|---|---|---|
| Linear Regression | 4.5986 | 6.6231 | 0.7512 | — |
| Ridge Regression | 4.5984 | 6.6229 | 0.7512 | α = 0.01 |
| **Random Forest** | **4.5713** | **6.5123** | **0.7594** | max_depth=10, n_estimators=200 |
| XGBoost | 4.5776 | 6.6307 | 0.7506 | lr=0.05, max_depth=5, n_estimators=100 |

Random Forest ยืนยันตำแหน่ง **Champion model** ด้วย MAE ต่ำที่สุดและ $R^2$ สูงที่สุด

---

# 4. Process and Teams

### 4.1 แนวทางการทำงาน

โปรเจกต์นี้ใช้รูปแบบ **Experiment-Driven + Iterative Development** ที่ผสมผสานแนวคิด Data Science กับ Software Engineering:

**Experiment-Driven:**

- ทุกการตัดสินใจ (model selection, feature engineering, threshold setting) ขับเคลื่อนด้วย experiment results
- เปรียบเทียบ 5 โมเดลอย่างเป็นระบบ (LR, Ridge, RF, XGBoost, LSTM) แล้วเลือก champion จาก MAE
- ผลลัพธ์ถูก track ใน MLflow สำหรับ reproducibility

**Iterative Development:**

- **Progress 1:** Baseline model (LR, Ridge, RF, XGBoost) กับข้อมูล 1 สถานี (daily Excel)
- **Progress 2:** เพิ่ม LSTM, Docker deployment, Airflow DAGs, monitoring, ขยาย dataset เป็น 86 สถานี
- **Progress 3:** อัพเดทผลลัพธ์กับข้อมูลใหม่ (station 145 hourly → daily), ยืนยัน pipeline robustness ข้ามสถานี

## 4.2 การทำงานร่วมกันระหว่าง Data Science และ Software Engineering

| ด้าน Data Science | ด้าน Software Engineering | จุดเชื่อมต่อ |
|---|---|---|
| Feature engineering design | API endpoint design | Feature columns JSON เป็น contract |
| Model training & evaluation | Docker containerization | Model artifacts (.joblib, .onnx) |
| Experiment tracking (MLflow) | CI/CD pipeline (Airflow) | DAG triggers training pipeline |
| Data drift analysis | Monitoring infrastructure | Threshold config ใน config.yaml |
| Model selection & promotion | API model loading | MODEL_NAME environment variable |

**การลด Communication Overhead:**

- ใช้ `configs/config.yaml` เป็น **single source of truth** — ทุก hyperparameter, threshold, path ถูกกำหนดที่จุดเดียว
- ใช้ **Interface/API** แยก concern: `src/api.py` เป็น boundary ระหว่าง ML logic กับ user-facing service
- ใช้ **Feature columns JSON** ( `models/feature_columns.json` ) เป็น contract ระหว่าง training กับ inference
- ใช้ **Docker Compose** ให้ทุก service สามารถ develop และ test อิสระจากกัน

## 4.3 การแบ่งบทบาท

| บทบาท | ความรับผิดชอบ |
|---|---|
| **ML Engineer** | Feature engineering, model training, hyperparameter tuning, experiment tracking |
| **Data Analyst** | Data Gathering, Feature Analysis, Result Declaration |
| **Data Engineer** | Data ingestion (scraper), preprocessing pipeline, Airflow DAGs |
| **Backend Engineer** | FastAPI service, Docker setup, ONNX export, monitoring module |
| **DevOps** | Docker Compose orchestration, PostgreSQL setup, MLflow server config |

## 4.4 ความท้าทายและการแก้ไข

| ความท้าทาย | การแก้ไข |
|---|---|
| LSTM ให้ผลแย่กว่า RF (MAE 6.22 vs 4.57) | Experiment-driven: ใช้ผลจริงตัดสินใจ ไม่ใช่ความคาดหวัง |
| ข้อมูลหลายรูปแบบ (hourly CSV vs daily Excel) | Data loader abstraction: แปลงเป็น `[date, pm25]` format เดียว |

| ความท้าทาย | การแก้ไข |
|---|---|
| Model serving ต้อง consistent กับ training | Feature columns JSON เป็น contract; ONNX ensure portability |
| Monitoring threshold ที่เหมาะสม | Iterative tuning: MAE 6.0 (จาก baseline MAE ~5.1), PSI 0.2 (standard threshold) |

---

# 5. Responsible ML Engineering

## 5.1 Reproducibility & Versioning

**Reproducibility:**

- `random_state = 42` กำหนดใน config.yaml ใช้ทุก model training
- `TimeSeriesSplit` (n_splits=3) สำหรับ cross-validation — ไม่ใช้ random split เพราะจะเกิด data leakage
- Temporal split เคร่งครัด: train ก่อน 2025, test หลัง 2025 — ไม่มี future data leak
- Feature engineering pipeline deterministic: lag, rolling, time features ให้ผลเดิมจาก input เดิม
- Experiment results บันทึกใน `results/experiment_results_station145.csv` สำหรับตรวจสอบ

**Versioning:**

- **Code:** Git + GitHub — ทุก commit tracked
- **Model:** MLflow experiment tracking — log params, metrics, artifacts ทุก training run
- **Data:** แยกไฟล์ per station ( `station_XX_long.csv` ) + archived originals ( `data/raw/archived/` )
- **Config:** `configs/config.yaml` version-controlled ร่วมกับ code

## 5.2 Explainability

**โมเดลสามารถอธิบายผลลัพธ์ได้:**

- Random Forest (Champion) สามารถดึง `feature_importances_` ได้โดยตรง → เห็นว่า lag_1 (ค่าเมื่อวาน) มีน้ำหนักสูงสุด ซึ่งสอดคล้องกับธรรมชาติของ PM2.5 ที่มี autocorrelation สูง
- Feature ทั้ง 17 ตัว **มีความหมายทางกายภาพชัดเจน** (lag = ค่าในอดีต, rolling = แนวโน้ม, time = ฤดูกาล) ไม่ใช่ black-box embedding
- API response แสดง `model` name + `prediction_date` → ผู้ใช้รู้ว่าใช้โมเดลอะไร ทำนายวันไหน
- **ข้อจำกัด:** ยังไม่มี per-prediction explanation (เช่น SHAP values) ใน API response → planned for future

**ผู้ใช้เข้าใจ prediction ได้:**

- Output เป็นค่าตัวเลขเดียว (μg/m³) + หน่วยชัดเจน → ตีความง่าย
- สามารถเทียบกับมาตรฐาน AQI ได้ทันที (< 25 = ดี, 25–37 = ปานกลาง, > 37 = เริ่มมีผลกระทบ)

## 5.3 Fairness

**ความเสี่ยงเรื่อง Bias:**

- **Geographic bias:** ปัจจุบันทดสอบ 2 สถานี (10T บางกะปิ, 145 บางขุนเทียน) — สถานีในพื้นที่ชนบทหรือริมอุตสาหกรรมอาจมี pattern ต่างกัน
- **Temporal bias:** Train data จาก 2023–2024 อาจไม่ represent สถานการณ์ผิดปกติ (เช่น ไฟป่า, El Niño)
- **Sensor quality bias:** สถานีบางแห่งมี missing data มากกว่า (เช่น station 145 มี 191 missing hours) → forward-fill อาจให้ค่าไม่ accurate ในช่วง missing ยาว

**การพิจารณากลุ่มผู้ใช้:**

- ระบบเป็น public service → ทุกคนเข้าถึงข้อมูลเดียวกัน
- ไม่มี personalization ที่อาจนำไปสู่ differential treatment
- แต่สถานีที่ได้รับการ tune โมเดลมีจำกัด → ผู้ใช้ในพื้นที่อื่นอาจได้ความแม่นยำต่ำกว่า

## 5.4 Safety

**ผลกระทบเมื่อ model ทำนายผิด:**

| สถานการณ์ | ผลกระทบ | ระดับความรุนแรง |
|---|---|---|
| Underpredict (ทำนายต่ำ แต่จริงสูง) | ผู้ใช้ออกกำลังกายกลางแจ้งในวันอากาศเสีย | **สูง** — เสี่ยงสุขภาพ |
| Overpredict (ทำนายสูง แต่จริงต่ำ) | ผู้ใช้งดกิจกรรมโดยไม่จำเป็น | **ต่ำ** — เสียโอกาส |

**แนวทางลดความเสี่ยง:**

- **Conservative thresholds:** ใช้ MAE 6.0 μg/m³ เป็น retraining trigger (เข้มงวดกว่า baseline MAE ~5.1)
- **Daily monitoring:** Airflow DAG ตรวจ performance ทุกวัน — ไม่ปล่อยให้ model เสื่อมนาน
- **Fallback strategy:** ถ้า API ไม่พร้อม → ผู้ใช้ยังดูค่าจริงจาก AirBKK ได้
- **Planned:** เพิ่ม confidence interval ใน prediction → ถ้า uncertainty สูง แสดงคำเตือน

## 5.5 Security & Privacy

**ข้อมูลอ่อนไหว:**

- **ไม่มีข้อมูลส่วนบุคคล** — ข้อมูล PM2.5 เป็น public environmental data
- ไม่มี user authentication/tracking → ไม่เก็บข้อมูลผู้ใช้

**แนวทางปกป้องข้อมูล:**

- API input validation: Pydantic schema ตรวจ `pm25` อยู่ในช่วง 0–500 → ป้องกัน injection
- API ไม่ expose internal model weights หรือ training data
- Docker network isolation: internal services (Airflow, MLflow, Postgres) ไม่เปิด public port
- **ข้อควรระวัง:** Airflow/MLflow web UI ควรมี authentication ที่แข็งแกร่งกว่า default admin/admin ใน production

## 5.6 Transparency & Accountability

**ระบบอธิบายข้อจำกัดได้:**

- `/model/info` endpoint แสดงชื่อโมเดล + feature list → ผู้ใช้รู้ว่าระบบใช้อะไร
- Experiment results เปิดเผยใน `results/experiment_results_station145.csv` → ตรวจสอบได้
- Monitoring results บันทึกใน `results/monitoring_results.csv` → audit trail
- **ข้อจำกัดที่ระบุชัดเจน:**
- ทำนายได้เฉพาะ **1 วันล่วงหน้า** (ไม่ใช่ 7 วัน)
- ต้องมีข้อมูลย้อนหลัง **≥ 15 วัน** จึงจะ predict ได้
- MAE ~4.5 μg/m³ หมายถึงค่าทำนายอาจคลาดเคลื่อน ±4.5 μg/m³
- ยังไม่รวม meteorological features (Temp, RH, Wind) → มีจุดปรับปรุง

**Accountability:**

- ทีมพัฒนารับผิดชอบในการ monitor model performance อย่างต่อเนื่อง
- Retrain trigger อัตโนมัติช่วยให้ model ไม่เสื่อมโดยไม่มีใครสังเกต
- MLflow log ทุก training run → สามารถ rollback ไปยัง model version ก่อนหน้าได้

---